



Universidad Autónoma de Nayarit

Unidad Académica De Economía

Sistemas Computacionales

“SEMANA 1: Introducción y Análisis de Complejidad”

Jorge Armando Navarrete Ruiz

Docente.- DR. ELIGARDO CRUZ SANCHEZ

U. A. ESTRUCTURA DE DATOS BÁSICA

COMPRENDE

Explorador de Complejidades

Complejidad	Analogía Personal	Patrón de Código que lo Genera	Situación donde sea visto o donde se pueda observar
O(1) Constante	Sacar el primer libro de una pila alta: no importa cuántos libros haya, el gesto es el mismo y toma prácticamente el mismo tiempo.	function obtenerPrimero(lista): if lista.esVacia(): return null return lista[0]	Chequear si una variable booleana está activa, leer longitud almacenada en una estructura (si se guarda). Útil para operaciones de control y verificaciones rápidas.
O(log n) Logarítmica	Buscar una palabra en la guía telefónica usando la estrategia «abrir a la mitad y decidir izquierda/derecha». Cada paso reduce el espacio de búsqueda a la mitad.	function busquedaBinaria(arr, clave): izq = 0; der = arr.length - 1 while izq <= der: mid = (izq + der) // 2 if arr[mid] == clave: return mid if arr[mid] < clave: izq = mid + 1 else: der = mid - 1 return -1	Búsqueda binaria en arrays ordenados, operaciones en árboles balanceados (B-tree, AVL, RB-tree) para búsqueda/insert/delete, búsqueda en tablas ordenadas, algunas operaciones de bases de datos con índices.
O(n) Lineal	Revisar cada carta en un mazo para encontrar una carta concreta: si hay el doble de cartas, tomará el doble de tiempo.	function maximo(arr): max = -INF for i from 0 to arr.length-1: if arr[i] > max: max = arr[i] return max	Escaneos de listas (buscar, sumar, filtrar), transformaciones por elemento (map/reduce), validaciones que deben mirar todos los datos, procesamiento en streaming.

$O(n \log n)$ Lineal Logarítmica	Ordenar libros por autor usando la estrategia de: dividir la pila en montones, ordenar cada montón y luego mezclar montones ordenados (como merge sort). Cada nivel de combinación procesa todos los libros, y hay $\approx \log n$ niveles.	<pre>function mergeSort(arr): if arr.length <= 1: return arr mid = arr.length // 2 left = mergeSort(arr[0:mid]) right = mergeSort(arr[mid:end]) return merge(left, right) # merge linear en tamaño</pre>	Ordenación de grandes colecciones (sort de arrays), algoritmos de procesamiento en árboles donde en cada nivel haces trabajo lineal (ej.: algunas transformaciones de datos), construcción de heaps, algunas fases de algoritmos de geometría.
$O(n^2)$ Cuadrática	En una fiesta, cada persona se saluda con cada otra persona una vez — si duplicas asistentes, las combinaciones de saludos crecen aproximadamente por el cuadrado.	<pre>function todosLosPares(arr): for i from 0 to arr.length-1: for j from i+1 to arr.length-1: procesar(arr[i], arr[j]) # operación constante</pre>	Algoritmos ingenuos de comparación de todos contra todos: detección de duplicados por comparación, generación de pares, algunas implementaciones sencillas de algoritmos en grafos (matriz de adyacencia con doble recorrido), versiones no optimizadas de matching o clustering para pequeños n .
$O(2^n)$ Exponencial	Probar todas las combinaciones posibles de encender/apagar n interruptores: cada interruptor tiene 2 estados, así que las combinaciones	<pre>function generarSubconjuntos(arr): if arr.empty(): return [[]] first = arr[0]; rest = arr[1:] subs = generarSubconjuntos(r</pre>	Problemas combinatoriales tratados por fuerza bruta: subset-sum, travelling salesman con fuerza bruta, backtracking sin

	son $2 * 2 * \dots = 2^n$. Con cada interruptor extra, las posibilidades se duplican.	est) return subs + [s + [first] for s in subs] # duplica subconjuntos	poda en peor caso, generación de todas las configuraciones posibles. Normalmente sólo tractable para n muy pequeño; en la práctica se busca poda (branch-and-bound), programación dinámica o heurísticas para reducir el espacio.
--	--	--	---

¿Cuál fue la analogía que más te ayudó a entender y por qué?

Fueron 2, La analogía Constante $O(1)$, y la Exponencial $O(2^n)$, Más que nada fue lo simples que son de tratar a la hora de aplicarlas, en sí, la constante mantiene una consistencia, mientras la exponencial aumenta “**n**” veces sea necesario, siendo que se duplique las veces que sean necesarias.

El Caso de DataStream Inc.

Diagnóstico escrito del problema de DataStream

El algoritmo actual se comporta super linealmente (exponente $\alpha \approx 1.2-1.6$; alrededor de 1.4 si partimos de $T_1 \approx 10$ s), por eso al crecer 50x el tiempo creció $\approx 282\times$.

Triplicar el hardware solo dio un 20% porque solo $\approx 30\%$ del trabajo es realmente paralelizable, o está limitado por I/O/contención. Agregar más Ram no arregla la complejidad si el cuello de botella es algoritmo/serialización.

Por lo que se deduce que la competencia usa un algoritmo Lineal o Lineal Logarítmico bien optimizado $O(n)$ o $O(n \log n)$ con constantes pequeñas y paralelización.

Cálculos que demuestren tu análisis de la complejidad probable.

Modelo: asumimos $T(n) = C \cdot n^\alpha$ (modelo polinómico general). Si al aumentar n por factor f , el tiempo cambia por factor $R = T_2/T_1 = f^\alpha$. Entonces $\alpha = \log(R) / \log(f)$.

Tomemos casos representativos para T_1 (porque “segundos” no es exacto):

Caso A: $T_1 = 5$ s $\rightarrow R = 2820 / 5 = 564$

$$\alpha = \ln(564) / \ln(50) = 6.334 / 3.912 = 1.62$$

Caso B (representativo): $T_1 = 10$ s $\rightarrow R = 2820 / 10 = 282$

$$\alpha = \ln(282) / \ln(50) = 5.642 / 3.912 = 1.44$$

Caso C: $T_1 = 30$ s $\rightarrow R = 2820 / 30 = 94$

$$\alpha = \ln(94) / \ln(50) = 4.543 / 3.912 = 1.16$$

Interpretación:

α está entre ≈ 1.1 y ≈ 1.6 dependiendo de la suposición de T_1 . Si tomamos $T_1 \approx 10$ s (muy plausible cuando dices “en segundos”), $\alpha \approx 1.44$.

Comparaciones con clases comunes:

- $O(n) \rightarrow R$ esperable $\approx f = 50$ (muy inferior a 282).
- $O(n \log n) \rightarrow R = f \cdot (\log n_2) / (\log n_1)$. Con $n_1=10k$, $n_2=500k$: $R \approx 50 \cdot (\ln 500000) / (\ln 10000) \approx 50 \cdot 1.424 \approx 71$ (muy inferior a 282).
- $O(n^2) \rightarrow R = f^2 = 2500$ (mucho mayor que 282).

Conclusión: el algoritmo actual es claramente superlineal ($\alpha > 1$) pero sub-cuadrático ($\alpha < 2$). Un ajuste razonable por los números es $\alpha \approx 1.3-1.6$ (en torno a 1.4 si $T_1 \approx 10$ s). Esto sugiere comportamientos como:

- pasos anidados parcialmente (ej.: bucle exterior sobre n y bucle interior que depende en promedio de n^β con $0 < \beta < 1$), o

- operaciones de ordenamiento repetidas o combinaciones costosas por grupo, o
- joins/agrupaciones no indexadas que en la práctica se comportan con coste $\approx n^{\{1.3-1.6\}}$ por la estructura de datos.

Una recomendación concreta: ¿Qué complejidad deberían buscar y por qué?

Deberían buscar la complejidad $O(1)$ y $O(n \log n)$; Con esto pueden mejorar la eficiencia profundamente sin modificar el hardware o añadir más ram, al designar $O(1)$ se hace una mejora en la eficiencia del DataStream, mientras que con $O(n \log n)$ se pueden verificar los datos y así mantener un orden en búsquedas y resultados si se pueden eficientar usando las dos complejidades.

Checkpoint Metacognitivo - Fase COMPRENDE

¿Puedo explicar con mis propias palabras qué mide Big O sin mencionar "tiempo de ejecución en segundos"?

Mide en cierta forma el tamaño en que crecen o se comportan los algoritmos dependiendo de a donde apunta su variable.

Si alguien me dice "mi algoritmo es $O(n^2)$ ", ¿qué imagen mental se forma? ¿Puedo predecir su comportamiento?

Puede ser algún objeto o suceso que se repitió dos veces, Si puedo predecir su comportamiento, ya que este tiende a duplicarse o en su caso a elevarse por sí mismo.

¿Qué preguntas formulé a la IA que no había considerado antes de comenzar?

Como consideraba la implementación de la complejidad constante y la lineal logarítmica. Fue una duda que surgió en razón a dar una respuesta.

¿En qué momento sentí que "esto tiene sentido" y qué lo provocó?

Fue cuando empecé a ver los resultados que daban cada una de sus complejidades, en sí, el cómo funciona cada una individual y en conjunto.

APLICA

Analizador de Código

Captura o registro de tu análisis de los 6 fragmentos con tu razonamiento escrito.

Fragmento 1

```
función obtenerPrimero(arr):  
    si arr.tamaño == 0:  
        devolver null  
    devolver arr[0]
```

Fragmento 2

```
función sumaElementos(arr):  
    total = 0  
    para i desde 0 hasta arr.tamaño - 1:  
        total = total + arr[i]  
    devolver total
```

Fragmento 3

```
función contarParesProducto(arr):  
    contador = 0  
    para i desde 0 hasta arr.tamaño - 1:  
        para j desde 0 hasta arr.tamaño - 1:  
            si (arr[i] * arr[j]) % 2 == 0:  
                contador = contador + 1  
    devolver contador
```

Fragmento 4

```
función buscarBinario(arr, objetivo):  
    inicio = 0  
    fin = arr.tamaño - 1  
    mientras inicio <= fin:  
        medio = (inicio + fin) // 2  
        si arr[medio] == objetivo:  
            devolver medio  
        si arr[medio] < objetivo:  
            inicio = medio + 1  
        sino:  
            fin = medio - 1  
    devolver -1
```

Fragmento 5

```
función aparenteCuadratica(arr):  
    i = 0  
    contador = 0  
    mientras i < arr.tamaño:  
        j = 0
```

```

    mientras j < arr.tamaño:
        contador = contador + 1
        j = j + 1
        i = i + 1
    devolver contador

```

Fragmento 6

función mezclaCompleja(arr):

```

    n = arr.tamaño
    // Paso A: recorrido lineal
    suma = 0
    para i desde 0 hasta n - 1:
        suma = suma + arr[i]

    // Paso B: bucle anidado con búsqueda binaria
    para i desde 0 hasta n - 1:
        // búsqueda binaria en la sublista arr[0..n-1]
        inicio = 0
        fin = n - 1
        mientras inicio <= fin:
            medio = (inicio + fin) // 2
            si arr[medio] == arr[i]:
                romper
            si arr[medio] < arr[i]:
                inicio = medio + 1
            sino:
                fin = medio - 1

    // Paso C: bucle con crecimiento geométrico
    j = 1
    mientras j < n:
        para k desde 0 hasta j - 1:
            hacerAlgunaOperacionConstante()
        j = j * 2

    // Paso D: doble bucle triangular
    para i desde 0 hasta n - 1:
        para j desde 0 hasta i:
            hacerOtraOperacionConstante()

    devolver suma

```

Para cada uno, documenta: tu hipótesis inicial, el razonamiento paso a paso, y la conclusión final.

Fragmento 1

Mi hipótesis inicial fue $O(n)$, confundí la terminología y lo que hacía, pero me hizo correcciones la IA con preguntas y una de ellas fue para darme cuenta que n era independiente del tamaño, por eso se ejecutaba de 1 en 1.

Mi respuesta final fue **$O(1)$** .

Fragmento 2

En el segundo mi primera respuesta fue $O(\log n)$ porque confundí con que reducía por mitades el arreglo al no leerlo cuidadosamente. Luego de dar mi respuesta me dieron 5 preguntas con lo que concluí con:

1.- es decreciente, se resta de 1 en 1, por eso el -1. 2.- $f(n)$ es igual a cuantas veces sea dado el valor de n . 3.- Si n es igual a 4 son 4 interacciones, si $n=100$ son 100 interacciones. 4.- Si se duplica n se agregan más interacciones. 5.- Me equivoque, creo que la complejidad más acertada a esto puede ser $O(n)$

$O(n)$ fue mi respuesta porque al duplicar o incrementar n aumentaban las interacciones.

Fragmento 3

Al iniciar con esta complejidad mi primer razonamiento fue este: Muy bien, lo que veo es un arreglo de dos dimensiones, este arreglo disminuye de 1 en 1, dependiendo de $n = \text{arr.tamaño}$ es cuánto sea el valor del contador, pero los arreglos son divididos en mod, por eso es el residuo de la mitad del arreglo, entonces en este caso es $O(\log n)$

Por lo que luego de obtener varias preguntas mi razonamiento final fue: 1.- si tomo $i=0$ entonces $j=n$, n es el razonamiento independiente, entonces j e i no son dependientes. 2.- en ese caso si $n=4$ entonces por cada vez que i incremente j va a tener 4 ejecuciones, así dando 16. 3.- $10 \cdot 10 = 100$. 4.- multiplicación pero también exponentes, se multiplican por sí mismos. 5.- no, hace que se incremente las veces que se ejecuta cada resultado del bucle. entonces si no estoy mal es $O(n \cdot n)$

Así mi respuesta final fue **$O(n^2)$**

Fragmento 4

Aquí mi razonamiento fue simple y directo al ya haber tenido varios errores pude acertar rápidamente con este razonamiento. La función busca reducir el arreglo por la mitad, puede ser el $O(\log n)$, me baso en esto por su división y por su formato a la hora de declarar todo con forma de reducir las búsquedas.

Mi respuesta final fue **$O(\log n)$**

Fragmento 5

Aquí mi razonamiento fue directo, por lo que pude responder rápido el código: mientras el arreglo sea mayor que i y que j se va a seguir ejecutando, el contador aumenta de 1 en 1, al igual que j e i , entonces el programa busca de uno en uno pasando por su arreglo hasta encontrar el número que cierre la función, entonces puede ser un $O(n)$

Mi respuesta final fue **$O(n)$**

Fragmento 6

Mi respuesta inicial fue notar que se combinaron varias complejidades, entre ellas $O(\log n)$, **$O(\log n)$** , **$O(n)$** y **$O(n^2)$** , luego de unas preguntas descubrí que el dominante en el código es $O(n^2)$.

Mi respuesta final fue **$O(n^2)$**

Identifica cuál te costó más trabajo y por qué.

Fue el fragmento 3, su código se me hizo muy confuso, al igual que confundí las complejidades y al momento de dar mis respuestas a las preguntas confundidas que hacen ciertos fragmentos del código y por eso mi interpretación de la complejidad era errónea.

Generador de Casos de Límite

Tabla comparativa con los cálculos de operaciones para cada caso.

Escenario	n	Posición P	buscarLineal — comparaciones arr[i]==objetivo	buscarDoble — iteraciones del cuerpo (i==j checks)	Ratio (B / A, aprox.)
1	100	50	$P+1=51$	$p \cdot n + (p+1) = 50 \cdot 100 + 51 = 5,051$	$5,051 / 51 \approx 99\times$
2	10,000	5000	$p+1 = 5,001$	$5,000 \cdot 10,000 + 5,001 = 50,005,001$	$50,005,001 / 5,001 \approx 10,000\times$
3	1,000,000	999,999	$p+1 = 1,000,000$	$999,999 \cdot 1,000,000 + 1,000,000 = 1,000,000,000,000 (\approx n^2)$	$1,000,000,000,000 / 1,000,000 = 1,000,000\times$

Explicación de 100 palabras sobre por qué códigos "que funcionan igual" pueden tener rendimientos radicalmente diferentes.

Aunque dos programas hagan lo mismo, no siempre tardan lo mismo en ejecutarse. La razón es que pueden tener estructuras distintas: uno puede recorrer los datos una sola vez y otro hacerlo muchas veces. En problemas pequeños ambos parecen rápidos, pero cuando la cantidad de datos crece, esa diferencia se nota mucho: **n frente a n^2** cambia radicalmente el tiempo de ejecución.

Además, influyen otros detalles como los costes fijos, la forma en que la memoria se usa (localidad), o cómo el procesador ejecuta el código. Sin embargo, lo más importante es la complejidad del algoritmo, porque es lo que determina cómo crecerá el rendimiento cuando los datos aumenten, incluso si la lógica final es la misma.

Propuesta de cómo detectarías este tipo de problemas en código real.

1. Determinar en cada código que complejidad de la Big O están usando para determinar cómo funcionan los loops usados.
2. Usar pruebas que determinen cómo pueden fallar los códigos en el peor de los casos.
3. Añadir ejemplos para que los programadores determinen los patrones que se puedan volver peligrosos.

Checkpoint Metacognitivo - Fase Aplica

¿Qué patrón de código es mi "señal" inmediata de $O(n^2)$? ¿Y de $O(\log n)$?

$O(n^2)$ Cuando son bucles anidados con la estructura for, éstos se ejecuta n veces sean requeridos. Una forma simple de decirlo, un arreglo de dos dimensiones o un For dentro de otro For.

$O(\log n)$ Cuando un código hace una división cada que se cumple su métrica busca reducir el espacio de búsqueda hasta que termine su ciclo

Cuando la IA corrigió mi análisis, ¿qué paso del razonamiento me había saltado?

Me falto identificar qué operación se repetía en cada ejecución del bucle.

¿Puedo ahora analizar código nuevo sin ayuda? Si no, ¿qué me falta?

No, aun me cuesta comprender ciertos patrones de las complejidades, ya que sigo confundiendo patrones y diferencias entre las Big O

¿Cómo cambiaron mis preguntas a la IA entre el Reto 3 y el Reto 4?

Una de las primeras fue que paso me salte al notar inconsistencias

Protocolo de Pensamiento en Voz Alta

Documento de reflexión con tus respuestas a las 5 preguntas.

Pregunta 1:

¿Cuál fue lo PRIMERO que miraste cuando viste el código? ¿Por qué empezaste ahí?

El punto 5, empecé ahí más que nada porque comprendo cómo funciona la división del mod y el div.

Pregunta 2:

¿En qué momento sentiste certeza sobre tu respuesta? ¿Qué te dio esa certeza?

No sentí certeza, solamente comprendí cómo puede funcionar el código desde mi lógica, entonces por eso decidí responder con lo que comprendo.

Pregunta 3:

Si te hubieras equivocado (o si te equivocaste), ¿cuál fue el supuesto que no verificaste?

Fue la comparación de $i == j$; no la evalué de la misma forma, eso fue el fallo en mi comprensión.

Pregunta 4:

¿Qué regla o patrón aplicaste casi automáticamente? ¿De dónde crees que viene ese automatismo?

Ejemplos vistos; he trabajado con C++ y otros lenguajes, me guío por lo que sé y por la experiencia que tengo, pero aun tengo fallas en mis comprensiones.

Pregunta 5:

Si tuvieras que enseñarle a un compañero cómo analizar este código, ¿cuáles serían tus 3 pasos?

1) Leer las variables; 2) pasar al cuerpo del código e interpretar qué hace cada función o esquema; 3) revisar las salidas de las variables para comprender qué se quiere obtener del ejecutable.

Tu "algoritmo personal" de 3-5 pasos para analizar la complejidad.

1. Detectar las variables de control y sus límites.

Apunta cada variable de bucle (como $i, j, k, \dots$) y hasta dónde llega en función de n .

2. Revisar si hay variables con dependencias entre bucles

Observar si alguna variable se modifica dentro de otro bucle o si hay condiciones que cambian cuántas veces se repite.

3. Calcular cuántas veces se ejecuta cada bucle

Para cada valor de la variable hay que determinar cuántas veces corre internamente.

Una debilidad identificada y un plan concreto para abordarla.

Tengo una tendencia a interpretar mal $i=j$.

Antes de declarar una complejidad tengo que analizar que controla el bucle, cuantas veces se ejecuta la variable interna por cada variable externa y analizar si hay algun cambio con estas variables o si son dependientes una de otra.

Conexión Teoría-Práctica

Mapa conceptual conectando: Concepto teórico → Aplicación profesional → Herramientas/Técnicas



3 situaciones de tu futura carrera donde aplicarás análisis de complejidad.

1. En la implementación para la optimización de tiempo a la hora de obtener datos de entrada a gran escala.
2. Determinar y dar recomendaciones a los usuarios dependiendo de su comportamiento en el código.
3. Detectar posibles manejos con fines maliciosos del código.

Lista de 2-3 herramientas de profiling/benchmarking que investigarás.

Profiling = descubrir "dónde" está el problema.

Benchmarking = medir "cuánto" rinde una implementación.

- Python: py-spy (profiling), timeit (benchmark simple).
- Java: async-profiler (profiling), JMH (benchmark).
- Load tests: wrk, k6.

Checkpoint Metacognitivo - Fase REFLEXIONA

¿Qué descubrí sobre mi propio proceso de pensamiento que no sabía antes?

Me cuesta procesar y diferenciar ciertas interacciones y ejecuciones en los códigos, pierdo el hilo cuando hay varios bucles dentro de otro bucle.

¿Cómo describiría mi "estilo" de análisis? ¿Soy más visual, más matemático, más intuitivo?

Soy más visual e intuitivo, necesito ver de forma gráfica y escuchar como debe de ser interpretado el código.

¿Qué conexión inesperada hice entre este tema y algo que ya sabía?

Son los bucles anidados al realizar arreglos de una y dos dimensiones.

Si empezara la semana de nuevo, ¿qué haría diferente en mi proceso de aprendizaje?

Leer e interpretar el código como también buscar ejemplos claros de cómo son interpretados los códigos.

El Auditor Implacable

Registro de los 4 casos con: tu identificación de errores, la corrección que propondrías, y verificación de si encontraste todos.

1. **Código 1: (arr): n = arr.tamaño para i desde 0 hasta n-1: j = n mientras j > 0: hacerOperacionConstante() j = j // 2 devolver**
 - **identificación de errores:** tratar el bucle interno ($j = j // 2$) como si fuera $O(1)$. La división entera por 2 genera interacciones.
 - **La corrección que propondrías:** Se debe reducir o eliminar el bucle, y también evitar repetir el proceso para cada i si se utiliza trabajo previo-
 - **Verificación de si encontraste todos:** Sí. Detectaste que no era $O(1)$ y que el término $\log n$ importa.
2. **Código 2: (arr): n = arr.tamaño para i desde 0 hasta n-1: para j desde 0 hasta n-1: si arr[j] == i: // hacer búsqueda binaria en arr ordenado index = busquedaBinaria(arr, i) devolver index**
 - **identificación de errores:** se asumió sin justificar que las llamadas a búsqueda binaria son “raras” y por tanto despreciables. Eso depende de los datos. El análisis omitió considerar cuántas veces se cumple.
 - **La corrección que propondrías:** Evitar tratar de llamar a búsqueda binaria dentro de un doble bucle.
 - **Verificación de si encontraste todos:** Parcialmente. Notaste confusión con $\log n$, pero no dejaste claro la suposición clave.
3. **Código 3: (n): para i desde 1 hasta n: j = i mientras j < n: hacerOperacionConstante() j = j + (i + 1) devolver**
 - **identificación de errores:** tras calcular correctamente la suma aproximada, el análisis decide “por seguridad” tomar $O(n^2)$. Eso es innecesariamente conservador e incorrecto como complejidad ajustada.
 - **La corrección que propondrías:** propuesta por copilot Razonamiento: por cada i el inner iteraciones $\approx n/(i+1)$. Sumando: $\sum_{i=1..n} n/(i+1) = n \cdot \sum_{i=1..n} 1/(i+1) = \Theta(n \log n)$.
 - **Verificación de si encontraste todos:** Sí. Identificaste que no era n^2 y propusiste la forma $n \log n$.
4. **Código 4: (arr): seen = conjunto vacío para x en arr: si x no está en seen: para y en seen: si y == x: romper añadir x a seen devolver**
 - **identificación de errores:** El análisis original dio $O(n^2)$. Eso es correcto en el peor caso. Sin embargo hay un matiz, la comprobación `if x not in seen` sugiere que se podría usar la operación de conjunto $O(1)$ para evitar todo el bucle interno; el código actual repite trabajo innecesario.
 - **La corrección que propondrías:** Eliminar el bucle interno
 - **Verificación de si encontraste todos:** Sí. Entendiste que el bucle interno puede crecer y que el coste puede ser cuadrático; además notaste el matiz de que la estructura `set` hace redundante el bucle y que una corrección baja a $O(n)$.

Porcentaje de errores detectados (meta: >75%).

- Detección completa: 3 de 4 $\rightarrow$ 75%.

- Si cuentas la detección parcial del caso 2 como medio acierto, entonces 3.5 de 4 → **87.5%.**
- Detalle por caso

Código 1: detectaste el error (interno = $\Theta(\log n)$, no $O(1)$) — acertado.

Código 2: detección parcial — notaste un problema con la interpretación de $\log n$, pero no señalaste la suposición clave del análisis original (que las llamadas a búsqueda binaria son “raras”); en el peor caso pueden ocurrir $\Theta(n^2)$ llamadas y entonces el coste llega a $O(n^2 \cdot \log n)$. Por eso lo marco como parcial.

Código 3: detectaste correctamente que es $\Theta(n \log n)$, no $\Theta(n^2)$. ♦♦ acertado.

Código 4: detectaste correctamente la cuadrática en el peor caso y el matiz de que se puede arreglar con el uso adecuado del set → acertado.

Categorización: ¿Qué tipo de errores fueron más difíciles de detectar?

El código 2 y código 3, en el segundo el código es difícil de interpretar, y en el tercero es fácilmente perdible el no notar cómo interpreta erróneamente el elevar al cuadrado los datos.

Simulación Empírica

Tabla de datos con tus cálculos de razones entre tiempos consecutivos.

n	Algoritmo A (ms)	Algoritmo B (ms)	Algoritmo C (ms)
100	2.060	0.490	2.684
500	9.400	13.125	17.035
1000	20.400	48.500	41.258
2000	42.000	212.000	85.104
5000	96.000	1187.500	260.399
10000	202.000	5400.000	520.878

Conclusión para cada algoritmo: ¿Los datos empíricos confirman la predicción teórica?

- **Algoritmo A:** los datos empíricos CONFIRMAN que A es aproximadamente $O(n)$. Tu juicio inicial (“no se asemeja a $O(n)$ ”) fue incorrecto; las razones muestran crecimiento lineal con ruido normal.

- **Algoritmo B:** los datos empíricos CONFIRMAN que B es $O(n^2)$. Tu observación de crecimiento muy rápido fue acertada, pero la idea de “elevado a la n (exponencial)” no está respaldada.
- **Algoritmo C:** los datos empíricos CONFIRMAN que C es $O(n \log n)$. Estás de acuerdo con esto y los datos lo apoyan.

Explicación del patrón esperado: "Si un algoritmo es $O(n^2)$, al duplicar n, el tiempo debería multiplicarse por ____".
por 4

Checkpoint Metacognitivo - Fase VALIDA

¿Qué tipo de error me costó más trabajo detectar en la auditoría? ¿Por qué crees que es así?

El código 2, sigo sin poder identificar de forma correcta ciertas partes anidadas del código

¿Confíé demasiado en alguna respuesta de la IA antes de verificarla? ¿Cómo puedo evitarlo?

Si, me deje llevar al leer el lenguaje claro pero también reflexivo que tiene, debo también asegurar mi comprensión de los temas y si tengo dudas entonces verificar por mi cuenta si eso es correcto o incorrecto.

¿Los datos empíricos me sorprendieron en algo? ¿Qué aprendí de esa discrepancia? Si, hay varias diferencias entre los datos que son entregados, si son vistos por muy encima se puede dar una conclusión errónea.

¿Cómo ha cambiado mi nivel de confianza en mis propios análisis después de esta fase?

Bastante, necesito corregir mis analisis varias veces como tambien debo verificar mis conocimientos para que sean acertados.